

Class-Based Verification Environment

For the 8-bit Combinational ALU

Presented to:

Analog Devices

Eng. Mohamed El Adawy

Presented by:

Mohamed Abdelhafez Elnewehy

July 2026

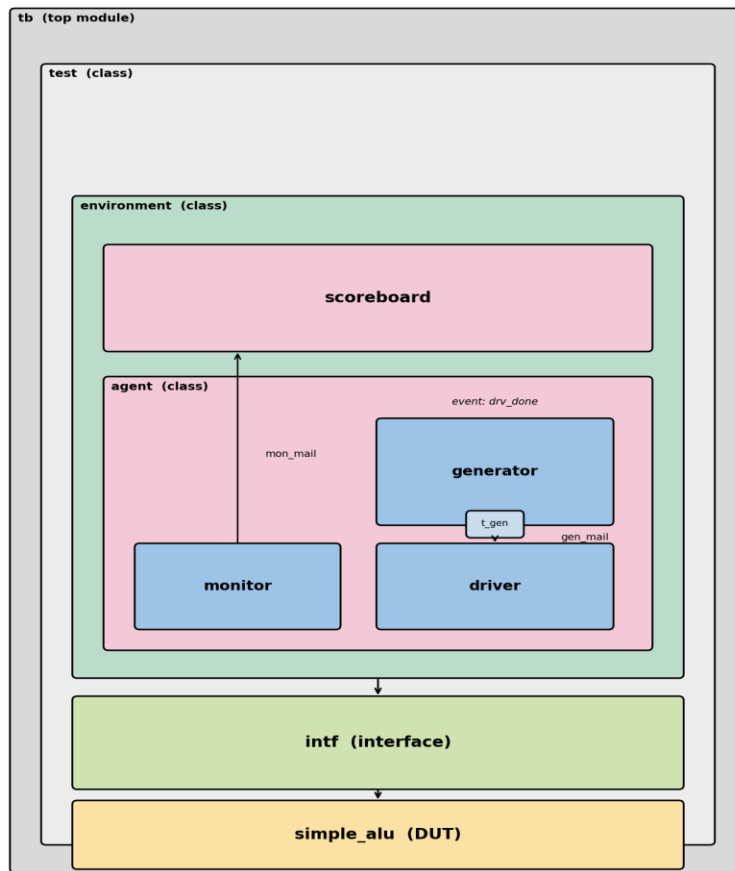
Verification Environment Report

1. Overview

This document explains the class-based verification environment built for the 8-bit combinational ALU. The environment follows a standard layered testbench architecture: a generator creates stimulus, a driver applies it to the DUT, a monitor observes the DUT's response, and a scoreboard checks that response against a reference model. All of these pieces are grouped inside an environment, which is started by a test, which in turn runs inside the top-level testbench module.

2. Architecture

The diagram below shows how the files map onto the standard testbench hierarchy, using the exact class and module names used in the code.



3. Component Descriptions

intf.sv

The SystemVerilog interface that connects the testbench to the DUT. It groups the ALU's signals (A, B, opcode, Result, Zero, Carry, Overflow) into a single bundle so that every class can access the DUT through one virtual handle instead of separate wires. It also carries a clk signal used only to pace the testbench, since the ALU itself is combinational.

transaction.sv

Defines the data item that moves through the environment. It holds the stimulus fields (A, B, opcode) as randomizable, and the response fields (Result, Zero, Carry, Overflow) that get filled in after the DUT reacts. A weighted constraint keeps most opcodes valid while still occasionally generating the two invalid codes. A copy() function lets the generator hand out an independent object for every iteration.

generator.sv

Produces the stream of transactions. The first fourteen iterations are directed, one for each corner case in the verification plan (carry-out, both signed-overflow directions, borrow, min/max shift amounts, zero flag, invalid opcodes). Every iteration after that is fully randomized. Each transaction is placed into `gen_mail`, and a `gen_handover` event marks that a new item is ready.

driver.sv

Takes each transaction out of `gen_mail` and drives A, B and opcode onto the interface. Since the ALU is combinational, the driver simply waits one time step for the logic to settle and then triggers `drv_done`, which tells the monitor it is safe to sample.

monitor.sv

Waits for `drv_done`, then reads the current values of A, B, opcode, Result, Zero, Carry and Overflow directly from the interface. It packages what it observed into a new transaction and forwards it to the scoreboard through `mon_mail`. The monitor never drives anything - it only watches.

scoreboard.sv

Receives each observed transaction from `mon_mail` and checks it against a golden reference model of the ALU, written independently inside the scoreboard itself. It recomputes the expected Result, Carry and Overflow for the same A, B and opcode, derives the expected Zero flag, and compares all four fields against what the DUT actually produced. Every check is logged as PASS or FAIL, and a running pass/fail count is kept for the final report.

agent.sv

A container that owns one driver and one monitor and runs them concurrently. It exists so the environment does not have to manage the driver and monitor individually - it simply asks the agent to run.

environment.sv

The top of the verification hierarchy. It creates the mailboxes (`gen_mail`, `mon_mail`) and the shared `drv_done` event, builds the generator, agent and scoreboard, wires them together, and launches all of them in parallel. It also exposes a `wait_for_end()` task that blocks until every transaction has been generated and checked, so the simulation knows when it is safe to stop.

test.sv

A thin wrapper around the environment. It builds one environment instance, starts it, and waits for it to finish before returning control to the testbench top module. Keeping this separate from `tb.sv` makes it easy to add other tests later without touching the top-level module.

tb.sv

The top-level module and the only piece of plain Verilog structure in the environment. It generates the pacing clock, instantiates the interface, instantiates the ALU DUT and connects it to the interface, and creates and runs the test. It ends the simulation once the test reports completion.

4. How a Transaction Flows Through the Environment

1. The generator creates a transaction (directed or random) and puts it into `gen_mail`.
2. The driver takes it from `gen_mail` and drives A, B and opcode onto the interface.
3. After allowing the combinational logic to settle, the driver triggers `drv_done`.
4. The monitor reacts to `drv_done` by sampling the DUT's outputs and sending a transaction to `mon_mail`.
5. The scoreboard takes the transaction from `mon_mail`, computes the expected values with its own reference model, and reports PASS or FAIL.
6. Once every transaction generated has been checked, the environment reports the final pass/fail count.

- Unsigned carry-out on ADD (e.g. $255 + 1$)
- Unsigned borrow on SUB (e.g. $0 - 1$)
- Signed overflow on ADD (e.g. $127 + 1$)
- Signed overflow on SUB (e.g. $-128 - 1$)
- Shift amount at its minimum (0) and maximum (7) for both SHL and SHR
- Zero flag asserted through AND
- Both invalid opcodes (110 and 111), confirming Result is forced to 0

```
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
# [MONITOR] A=103 B=166 opcode=001 | Result=193 Zero=0 Carry=1 Overflow=1
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
# [MONITOR] A=103 B=166 opcode=001 | Result=193 Zero=0 Carry=1 Overflow=1
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
# [MONITOR] A=103 B=166 opcode=001 | Result=193 Zero=0 Carry=1 Overflow=1
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
# [MONITOR] A=103 B=166 opcode=001 | Result=193 Zero=0 Carry=1 Overflow=1
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
# [MONITOR] A=103 B=166 opcode=001 | Result=193 Zero=0 Carry=1 Overflow=1
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
# [MONITOR] A=103 B=166 opcode=001 | Result=193 Zero=0 Carry=1 Overflow=1
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
# [MONITOR] A=103 B=166 opcode=001 | Result=193 Zero=0 Carry=1 Overflow=1
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
# [MONITOR] A=103 B=166 opcode=001 | Result=193 Zero=0 Carry=1 Overflow=1
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
# [MONITOR] A=103 B=166 opcode=001 | Result=193 Zero=0 Carry=1 Overflow=1
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
# [MONITOR] A=103 B=166 opcode=001 | Result=193 Zero=0 Carry=1 Overflow=1
# [SCOREBOARD] PASS | A=103 B=166 Op=1 -> Res=193
#
# =====
#                FINAL TEST REPORT
# =====
# PASSED: 6019
# FAILED: 0
# =====
```

This environment separates stimulus generation, DUT interaction and checking into independent classes that communicate only through mailboxes and a shared event. This keeps each piece simple and replaceable - the ALU's reference model, for example, can be updated in scoreboard.sv without touching the driver, monitor, or generator. The directed corner cases guarantee every requirement in the verification plan is exercised at least once, while the randomized iterations add broader confidence beyond the specific cases called out.